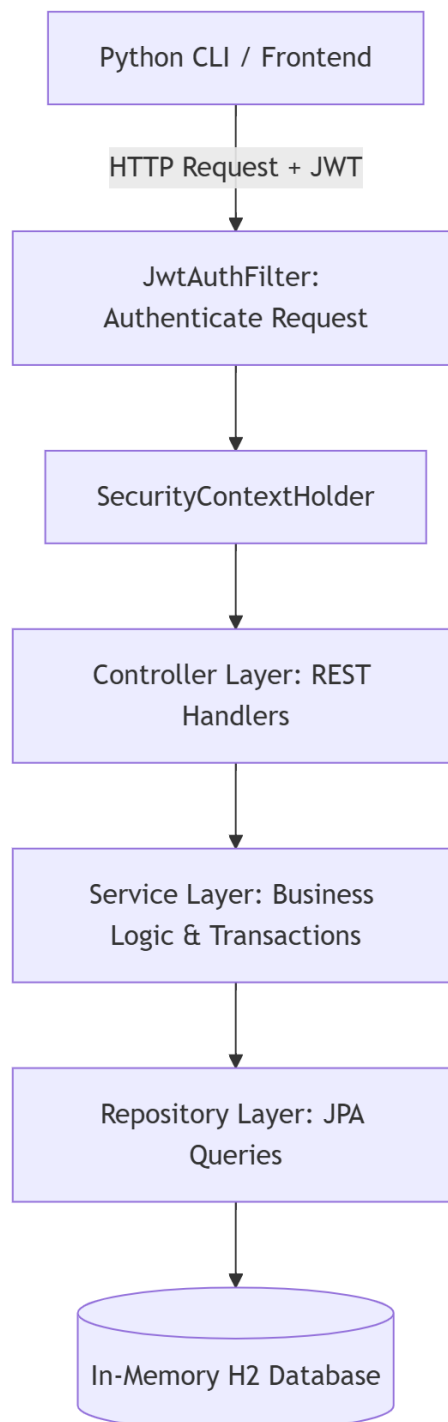


Pasadena City College Grade Portal - Backend Architecture Guide

This document describes the design, packages, data flow, and security implementation of the Java Spring Boot REST API backend application.

1. Architecture Overview

The backend is built as a **stateless, layered REST API** using Java 17, Spring Boot, Spring Security (JWT), Spring Data JPA, and an in-memory H2 database.



Layer Separation

1. **Presentation (Controllers):** Handles incoming HTTP requests, extracts parameters/bodies, delegates to services or repositories, and returns JSON responses.
2. **Security Filtering:** Intercepts requests to validate JWT signatures, extract scopes/roles, and populate the Spring Security authorization context.
3. **Business Logic (Services):** Coordinates transactions and business-specific tasks (e.g., CSV mapping, validation, logging).
4. **Data Access (Repositories):** Interface declarations extending `JpaRepository` to abstract SQL generation.
5. **Data Transfer Objects (DTOs):** Holds serialization schemas for API request and response bodies.

📁 2. Backend Package Directory

All files are nested under the primary package: `edu.pasadena.grademanager`

- `StudentGradeManagementSystemApplication.java`: Entrypoint bootstrap class.
- `config/`:
 - `SecurityConfig.java`: Registers filters, disables CSRF (stateless API context), configures CORS, and controls endpoint access matching.
- `security/`:
 - `JwtAuthFilter.java`: Pulls tokens from incoming authorization headers and populates `SecurityContext`.
 - `JwtUtils.java`: Utility functions to generate, sign, and validate JWTs.
 - `UserDetailsServiceImpl.java`: Connects user repository credentials check to Spring's core authentication.
- `controller/`:
 - `AuthController.java`: Handles logins, admin user creation, and password updates.
 - `StudentController.java`: Coordinates student profiles and handles sensitive field overrides.
 - `GradeController.java`: Manages assignment grade records and CSV uploads.
 - `UserController.java`: Manages authentication user directories (Admin-only).
- `model/`: JPA database mapping classes (`User` , `Student` , `Grade` , `Course` , `Assignment` , `Role`).
- `repository/`: `JpaRepository` interfaces mapping entities to standard database CRUD operations.
- `service/`: Handles multi-model database transactions (e.g. `CsvIngestionService`).
- `util/`:
 - `DataSeeder.java`: Seeds the system with test accounts on startup.

🔑 3. JWT Security & Filter Pipeline

The application operates in a completely stateless environment. No session cookie is generated. Instead, authentication relies on token exchange verified on each incoming request:

The Authentication Filter Flow

1. **Request Capture:** Every call hits `JwtAuthFilter` first before hitting any MVC controller.
2. **Token Extraction:** The filter reads the HTTP request header:
`Authorization: Bearer <token>`
3. **Signature Validation:** `JwtUtils` decodes the token, validating it against a secure secret signature.
4. **Context Injection:** On validation, the user details are loaded from the database and wrapped inside a `UsernamePasswordAuthenticationToken` which is then saved in Spring's `SecurityContextHolder`.

Endpoint Constraints

- **Permitted Endpoints:** Public routes (e.g. login `/api/auth/login`) are configured with `.permitAll()` inside `SecurityConfig`.
- **Role Requirements:** Handlers are guarded using either method security or context mapping:

```
@DeleteMapping("/{id}")  
@PreAuthorize("hasRole('ADMIN')")  
public ResponseEntity<> deleteStudent(...) { ... }
```
